

LABORATORIO DI PROGRAMMAZIONE
CORSO DI LAUREA IN SICUREZZA DEI SISTEMI E DELLE RETI
INFORMATICHE
UNIVERSITÀ DEGLI STUDI DI MILANO
2025–2026

INDICE

Parte 1. Esercizi con i puntatori	3
Esercizio 1	3
<i>Funzione reverse</i>	3
Tempo: 15 min.	3
Esercizio 2	4
<i>Funzione equals</i>	4
Tempo: 15 min.	4
Esercizio 3	4
<i>Funzione palindroma</i>	4
Tempo: 30 min.	4
Esercizio 4	5
<i>Confronto lessicografico tra stringhe</i>	5
Tempo: 25 min.	5
Esercizio 5	5
<i>Funzione clean</i>	5
Tempo: 20 min.	5
Esercizio 6	5
<i>Funzione clean parametrica</i>	5
Tempo: 10 min.	5
Esercizio 7	5
<i>Funzione clean parametrica, con più caratteri</i>	5
Tempo: 25 min.	6
Esercizio 8	6
<i>Ordinamento delle stringhe per lunghezza</i>	6
Tempo: 40 min.	6
Esercizio 9	6
<i>Ordinamento lessicografico delle stringhe</i>	6
Tempo: 20 min.	6
Esercizio 10	6
<i>Trova l'errore1</i>	6
Tempo: 20 min.	7
Esercizio 11	7
<i>Trova l'errore</i>	7

Tratti dagli esercizi del corso del prof. Vincenzo Marra.
Ultima revisione: 17 novembre 2025.

Tempo: 20 min.

Parte 1. Esercizi con i puntatori

Avvertenza

In questa lezione di laboratorio farete esercizi sull'uso dei puntatori a carattere per la manipolazione delle stringhe. Per la lettura delle stringhe inserite dall'utente userete sempre la funzione `fgets` della libreria standard. L'Esercizio 1 riporta il codice necessario per la lettura di una stringa con `fgets`. Se preferite, potete anche scrivere all'inizio dell'esercitazione una funzione ausiliaria che usi `fgets` per acquisire una stringa da tastiera, e poi riutilizzarla nei diversi esercizi. Nel caso vogliate acquisire un carattere inserito dall'utente, usate la funzione `getchar()` della libreria standard.

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      void reverse(char*, char*);
6
7      /* BUFSIZ e' una costante definita
8       in stdio.h */
9      char str1[BUFSIZ], str2[BUFSIZ];
10
11     printf("Inserisci una stringa: ");
12     if( fgets (str1, BUFSIZ, stdin)==NULL )
13     {
14         printf("Errore in lettura!\n");
15         return -1;
16     }
17     /* Sovrascrive \n finale */
18     int i=0;
19     for (;str1[i]!='\n';i++);
20     str1[i]='\0';
21
22     reverse(str1, str2);
23
24     printf("La stringa al contrario e':\n%s\n", str2);
25
26     return 0;
27 }

```

FIGURA 1. Esempio di implementazione della funzione `main` per l'Esercizio 1.

ESERCIZIO 1

Funzione reverse.

Tempo: 15 min.

Scrivete un programma che chieda all'utente di inserire una stringa, invochi una funzione che restituisce una copia della stringa scritta in ordine inverso, la visualizzi e termini. La funzione che inverte la stringa ha prototipo

```
void reverse(char *s, char *t)
```

dove il primo parametro rappresenta la stringa da invertire, e il secondo la stringa invertita. In `main`, invocate la funzione passandole come primo parametro (il nome di un) array di tipo `char` inizializzato con una costante stringa — ossia la stringa letta dall'utente — e come secondo parametro (il nome di un) array di tipo `char` della medesima dimensione del primo. Un esempio di come potrebbe essere scritta la funzione `main` del programma è in Fig. 1.

ESERCIZIO 2

Funzione `equals`.

Tempo: 15 min.

Scrivete una funzione di prototipo

```
int equals(char *s, char *t)
```

che accetti in ingresso due stringhe, e restituisca in uscita un intero di valore non negativo nel caso le due stringhe siano uguali, e nullo altrimenti. Scrivete una funzione `main` atta a testare adeguatamente la vostra funzione.

Come si comporta la vostra implementazione di `equals` se uno o entrambi i puntatori in ingresso valgono `NULL`?

ESERCIZIO 3

Funzione `palindroma`.

Tempo: 30 min.

Scrivete una funzione di prototipo

```
int palindroma(char *s)
```

che accetti in ingresso una stringa, e restituisca in uscita un intero di valore non negativo nel caso la stringa sia palindroma, e nullo altrimenti. Scrivete una prima versione che utilizzi le funzioni `reverse` e `equals` degli Esercizi 1 e 2. Scrivete una funzione `main` atta a testare adeguatamente la vostra funzione. Scrivete poi una seconda implementazione della funzione `palindroma` che utilizzi solo la stringa `s` passata in argomento, senza invocare funzioni ausiliarie e senza allocare memoria per una copia di `s`.

Come si comporta la vostra implementazione di `palindroma` se il puntatore in ingresso valgono `NULL`?

Prestate attenzione al fatto che la funzione `fgets` include nell'array di caratteri letti anche il carattere `'\n'` di terminazione riga: nell'esempio è incluso il codice necessario a sovrascrivere `'\n'` con `'\0'`.

Questa seconda implementazione è meno facile da scrivere, e può sembrare una soluzione meno elegante rispetto alla prima: osserverete, tuttavia, che essa è senz'altro la più efficiente delle due dal punto di vista dell'uso delle risorse fisiche a disposizione (spazio in memoria, tempo di calcolo).

ESERCIZIO 4

Confronto lessicografico tra stringhe.

Tempo: 25 min.

Scrivete una funzione che permetta di confrontare le stringhe secondo l'ordinamento lessicografico (cioè quello dei dizionari). Il prototipo è:

```
int lex(char *s, char *t)
```

I parametri in ingresso rappresentano le due stringhe da confrontare. La funzione restituisce un intero positivo se `s` precede strettamente `t` nell'ordinamento, un intero negativo se `t` precede strettamente `t` nell'ordinamento, e zero nel caso in cui le due stringhe siano uguali. Scrivete una procedura `main` appropriata che permetta di testare la vostra implementazione.

ESERCIZIO 5

Funzione `clean`.

Tempo: 20 min.

Scrivete un programma che chieda all'utente di inserire una stringa, invochi una funzione che restituisce una copia della stringa privata degli spazi, la visualizzi e termini. La funzione che elimina gli spazi dalla stringa ha prototipo

```
void clean(char *s, char *t)
```

dove il primo parametro rappresenta la stringa da ripulire, e il secondo la stringa ripulita. In `main`, invocate la funzione passandole come primo parametro (il nome di un) array di tipo `char` inizializzato con una costante stringa, e come secondo parametro (il nome di un) array di tipo `char` della medesima dimensione del primo.

ESERCIZIO 6

Funzione `clean` parametrica.

Tempo: 10 min.

Scrivete un programma che chieda all'utente di inserire una stringa, invochi una funzione che restituisce una copia della stringa privata di tutte le occorrenze di un carattere passato in argomento alla funzione, la visualizzi e termini. La funzione che elimina le occorrenze del carattere dalla stringa ha prototipo

```
void clean(char *s, char *t, char c)
```

dove il primo parametro rappresenta la stringa da ripulire, il secondo la stringa ripulita, e il terzo il carattere le cui occorrenze sono da eliminare. In `main`, invocate la funzione passandole come primo parametro (il nome di un) array di tipo `char` inizializzato con una costante stringa, e come secondo parametro (il nome di un) array di tipo `char` della medesima dimensione del primo.

Riutilizzate il codice che avete scritto per l'Esercizio 5.

ESERCIZIO 7

Funzione `clean` parametrica, con più caratteri.

Tempo: 25 min.

Riutilizzate il codice che avete scritto per l'Esercizio 6.

Scrivete un programma che chieda all'utente di inserire una stringa, invochi una funzione che restituisce una copia della stringa privata di tutte le occorrenze degli elementi di un insieme di caratteri passato in argomento alla funzione, la visualizzi e termini. La funzione che elimina le occorrenze dei caratteri dalla stringa ha prototipo

```
void clean(char *s, char *t, char *u)
```

dove il primo parametro rappresenta la stringa da ripulire, il secondo la stringa ripulita, e il terzo una stringa i cui caratteri sono quelli le cui occorrenze sono da eliminare (escluso il `\0` di terminazione di `u`). Per esempio, se `u` punta a `"A x"`, dalla stringa `s` saranno eliminate tutte le occorrenze dei caratteri `'A'`, `' '` (=spazio) e `'x'`. Nel `main`, invocate la funzione passandole come primo parametro (il nome di un) array di tipo `char` inizializzato con una costante stringa, come secondo parametro (il nome di un) array di tipo `char` della medesima dimensione del primo e come terzo parametro (il nome di un) array di tipo `char` inizializzato con una costante stringa.

ESERCIZIO 8

Ordinamento delle stringhe per lunghezza.

Tempo: 40 min.

Scrivete un programma che chieda all'utente di inserire 5 stringhe, e le memorizzi in un array di puntatori a `char` appropriatamente dimensionato. Il programma chiama poi una funzione di prototipo

```
void ordina(char **s, int lung)
```

il cui primo parametro rappresenta l'array di puntatori a `char`, e il cui secondo parametro rappresenta il numero di stringhe puntate dagli elementi dell'array (in questo esempio, 5). La funzione ordina l'array di stringhe in ordine non decrescente di lunghezza. Scrivete una funzione ausiliaria per il calcolo della lunghezza di una stringa. Come algoritmo di ordinamento usate Bubble Sort, che avete implementato nell'Esercizio 11 della Lezione 5.

ESERCIZIO 9

Ordinamento lessicografico delle stringhe.

Tempo: 20 min.

Modificate la soluzione dell'Esercizio 8 di modo che le stringhe inserite dall'utente siano ordinate secondo l'ordine lessicografico, invece che secondo la loro lunghezza. Per il confronto lessicografico fra stringhe, riutilizzate la funzione che avete scritto per risolvere l'Esercizio 4 della Lezione 6.

ESERCIZIO 10

Trova l'errore1.

```

1 void reverseEasy(char *s, char* srev){
2     int length, c;
3     char *end = s;
4     length = strlen(s);
5     for (c = 0; c < length - 1; c++)
6         end++;
7
8     for (c = 0; c < length; c++){
9         *srev = *end;
10        srev++;
11        end--;
12    }
13 }

```

FIGURA 2. Esempio di implementazione *sbagliata* della funzione `reverse` per l'Esercizio 1.

```

1 void reverse (char * s, char * t){
2     int i = 0, j = 0;
3     int cont = 0;
4     while(*(s+i) != '\0')
5         i++;
6     while((t[j] = s[i - 1])){
7         j++;
8         i--;
9         cont++;
10    }
11 }
12

```

FIGURA 3. Esempio di implementazione *sbagliata* della funzione `reverse` per l'Esercizio 1.

Tempo: 20 min.

Il codice in Figura 2 dovrebbe essere una possibile soluzione all'esercizio 1. Tuttavia, è sbagliato. Trovate l'errore e correggetelo.

ESERCIZIO 11

Trova l'errore.

Tempo: 20 min.

Il codice in Figura 3 dovrebbe essere una possibile soluzione all'esercizio 1. Tuttavia, è sbagliato. Trovate l'errore e correggetelo.